Artificial Neural Network-Based Model Predictive Control For Voltage Oscillations Stabilization in DC Microgrids

Ranjan Kumar[a], C. N. Bhende[a], Yash Raghuwanshi[b] and Nawaz Hussain[a]

[a]School of Electrical Sciences, Indian Institute of Technology, Bhubaneswar, 752050, India
[b]Energy Systems Innovation Center, Washington State University, Pullman, 99163, USA

ARTICLE INFO

Keywords:
active damping
artificial neural network
constant power load
DC microgrids
three phase inverter
model predictive control

ABSTRACT

To supply AC loads in a DC microgrid, the point-of-load (POL) converters integrated with the input LC filter are essential. Nevertheless, this integration introduces mismatch in input and output impedances at the DC interface leading to instability issues and significant voltage oscillations in the DC-link voltage. To address this issue, many researchers utilise finite control set model predictive control (FCS-MPC)-based techniques due to their active damping capablities, rapid response, and accurate tracking. However, FCS-MPC requires substantial real-time computation to be solved online. To overcome this challenge, this paper proposes a simple supervised imitation learning method. The proposed controller imitates the FCS-MPC scheme using labelled data, effectively shifting the majority of the computational load from online to offline platforms by utilising a trained artificial neural network (ANN) subject to robustness characteristics. Furthermore, parameter uncertainties resulting from reliance on the mathematical model of the system may lead to poor MPC design. This article presents a model based, parameter free control strategy that uses ANN to mitigate the negative effects of parameter discrepancies on POL converter performance.

1. Introduction

In a DC microgrid, point-of-load (POL) converters play a crucial role in delivering power from the common DC bus to various loads [1, 2]. As illustrated in Fig. 1, a typical DC microgrid with POL converters shows these converters connected to the DC bus. However, the high switching frequencies employed by POL converters can introduce electromagnetic interference (EMI) noise, which can propagate onto the common DC bus. To mitigate this EMI noise, it is standard practice to incorporate an input EMI filter, typically an LC filter, before the POL converter, as shown in Fig. 2. The tightly regulated nature of POL converters causes POL converter to behave as constant power loads (CPLs) [3]. This characteristic presents a significant challenge due to the negative input impedance associated with CPLs. The negative input impedance can lead to instability issues and substantial voltage oscillations across the DC capacitor, compromising the overall system's performance and reliability [4, 5].

1.1. Literature Review

Recently, substantial research have been conducted to address the voltage oscillations and make the system stable [1–7] through damping. Damping control is categorised as passive and active damping. In passive damping, physical passive elements such as R, L, and C, or combinations thereof, are required to damp voltage oscillations. However, this introduces additional costs and power losses [5]. In active damping, no extra element is needed; the damping element is virtually inserted by modifying the control loop/scheme. Active damping can be implemented using linear or non-linear controllers. In linear controllers, both the main converter circuit and the control feedback system are linearized at certain equilibrium points, which may fail during large disturbances [6]. Additionally, using non-linear controllers can address the aforementioned drawbacks. Among various non-linear controllers, the most popular is the FCS-MPC, due to its attractive features such as modulator independency, simplicity, rapid response, accurate tracking, and explicit handling of constraints.

Recent studies, particularly [2], [5], and [7], have investigated the implementation of a FCS MPC-based stabilising cost function to mitigate voltage oscillations across the DC capacitor (C_{dc}). These techniques primarily aim to control

rk53@iitbbs.ac.in (R. Kumar); cnb@iitbbs.ac.in (C.N. Bhende); y.raghuwanshi@wsu.edu (Y. Raghuwanshi); nawazhussain@iitbbs.ac.in (N. Hussain)
ORCID(s): 0000-0003-2193-6935 (R. Kumar); 0009-0002-7397-6461 (Y. Raghuwanshi)

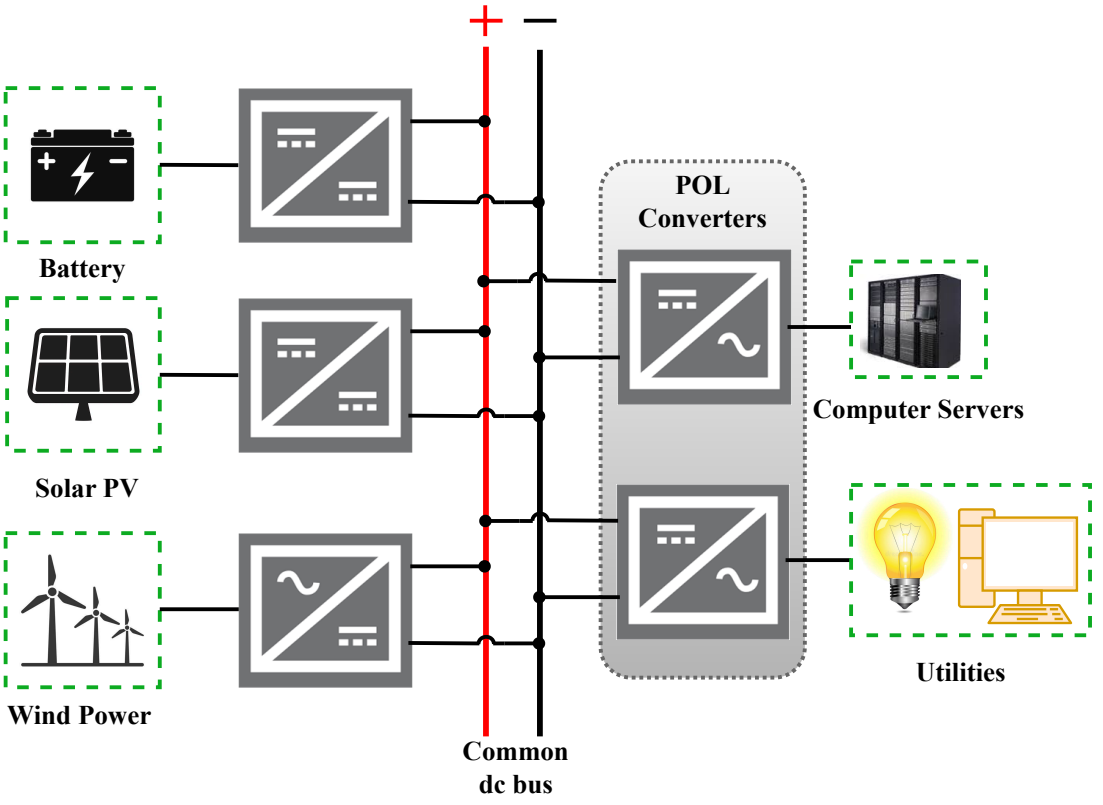


Fig. 1: Diagram of typical DC microgrids

the AC side voltage while concurrently stabilising the DC side voltage oscillations; DC voltage regulation is an extra objective that is integrated into the main control objectives along with suitable weighting factors. However, in these FCS-MPC-based works [2], [5], and [7], achieving high-precision tracking performance necessitates an accurate system model. Also, these models are consistently complicated in real-world circumstances by system uncertainties and external disturbances. Furthermore, the work presented in [7] gives high total harmonic distortion (THD) and takes a long time to damp the DC side voltage oscillation. Moreover, approach adopted in [5] affects the AC voltage regulation performance of the POL converter. A fundamental problem in FCS-MPC is that it requires the optimisation problem to be solved online, which involves a large number of real-time calculations, imposing a heavy computational burden [2, 8].

To this extent, the artificial neural network (ANN)-based approach has recently garnered significant attention in the field of power electronics [9–11]. In [9], an ANN-based FCS-MPC for a three-phase voltage source inverter was developed to reduce THD while improving steady-state and dynamic performance. Similarly, [10] proposed an ANN strategy for three-phase flying capacitor multi-level inverters, and [11] employed an ANN approach to tune MPC cost function weights. Being parameter free, ANNs treat the entire system as a black box. Importantly, it can be developed using real system data without requiring expert knowledge, utilising the end-to-end (E2E) learning capability to map raw inputs to desired outputs. The ANN is trained on a wide range of operating conditions and system behaviors, enabling it to capture the nonlinear dynamics and uncertainties inherent in DC microgrids. This data-driven approach allows the controller to adapt to variations in system parameters, load changes, and other disturbances without the need for precise mathematical models. Although significant training data is required, reliable simulations can generate the necessary data, mitigating this requirement [9, 11]. To the best of authors knowledge, no ANN method exists that can damp the DC voltage oscillations caused by CPL and improve the THD on the AC side. This is the motivation behind this work.

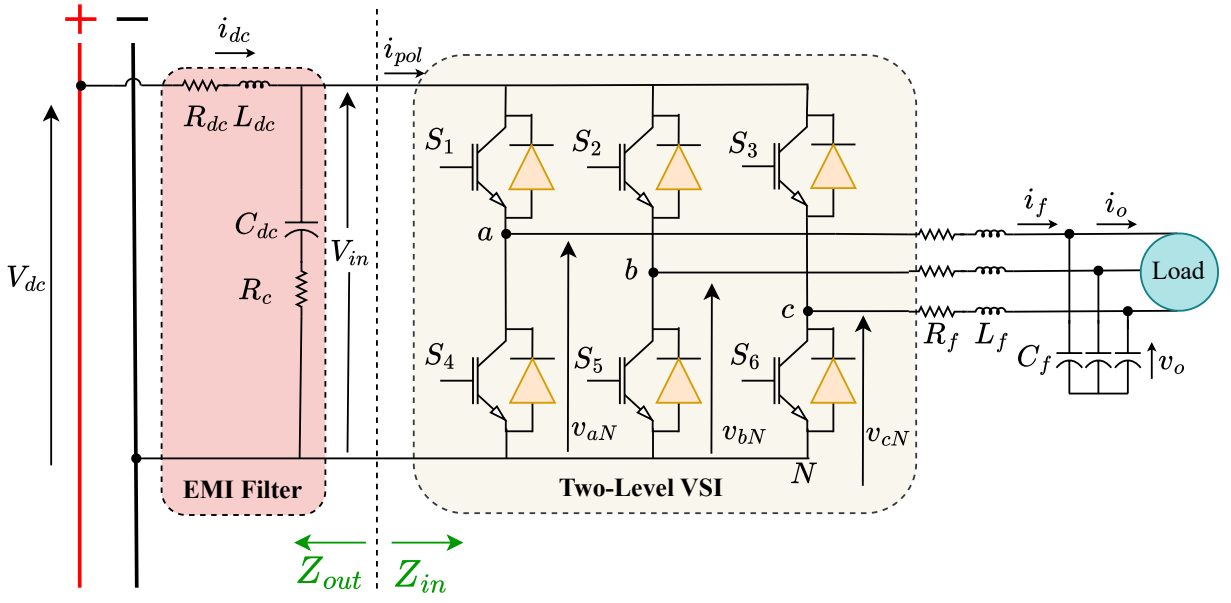


Fig. 2: Basic structure of two-level three-phase VSI

1.2. Main Contribution

Motivated by the aforementioned limitations associated with the state-of-the-art methods, this article proposes an ANN-based approach for stabilising voltage oscillations. The main contributions of this work are as follows:

1. The proposed methods address parameter mismatch issues by implementing a parameter-free control strategy using ANNs.
2. The THD of AC-side voltage is improved. Also, a short settling time to suppress voltage oscillations is achieved.
3. The online computational burden is alleviated through offline training of the ANN.

The sections of this article are organised as follows: Section II covers the fundamentals of FCS-MPC for a three-phase voltage source inverter (VSI). Section III provides details about the proposed ANN-based control strategy. The simulation results are presented in Section IV. Finally, Section V provides the paper's conclusion.

2. FCS-MPC for Three Phase VSI

This section begins by establishing the converter model of the VSI system, essential for understanding its performance and behavior. It details the VSI's operation, including switching mechanisms and control strategies. Following this, the section explores the dynamic model of an LC-filtered VSI.

2.1. Converter model of three phase VSI

In this paper, the topology of a three-phase, two-level VSI is used to emulate a CPL. As shown in Fig. 2, the VSI consists of three legs, with each leg containing two switches operated in a complementary manner to prevent short circuit conditions. The converter's switching states are determined by gating signals, S_a, S_b, and S_c, as follows:

Table 1
Possible switching states and voltage vectors for a two level three-phase VSI

S_a	S_b	S_c	Voltage vector v_i
0	0	0	$v_0 = 0$
1	0	0	$v_1 = \frac{2}{3}V_{in}$
1	1	0	$v_2 = \frac{1}{3}V_{in} + j\frac{\sqrt{3}}{3}V_{in}$
0	1	0	$v_3 = -\frac{1}{3}V_{in} + j\frac{\sqrt{3}}{3}V_{in}$
0	1	1	$v_4 = -\frac{2}{3}V_{in}$
0	0	1	$v_5 = -\frac{1}{3}V_{in} - j\frac{\sqrt{3}}{3}V_{in}$
1	0	1	$v_6 = \frac{1}{3}V_{in} - j\frac{\sqrt{3}}{3}V_{in}$
1	1	1	$v_7 = 0$

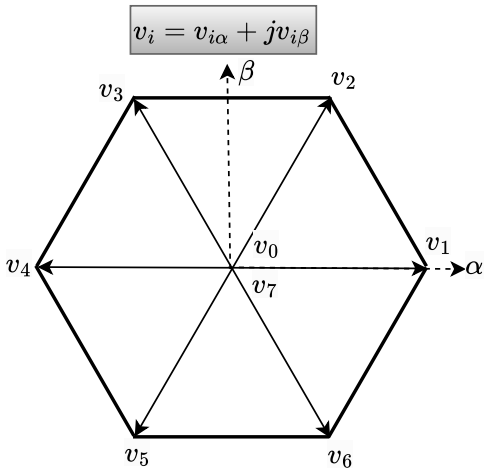


Fig. 3: Possible voltage vector in complex plane

$$
\begin{aligned}
S_a &= \begin{cases} 1, & \text{if } S_1\text{ON and } S_4\text{OFF} \\ 0, & \text{if } S_1\text{OFF and } S_4\text{ON} \end{cases} \\
S_b &= \begin{cases} 1, & \text{if } S_2\text{ON and } S_5\text{OFF} \\ 0, & \text{if } S_2\text{OFF and } S_5\text{ON} \end{cases} \\
S_c &= \begin{cases} 1, & \text{if } S_3\text{ON and } S_6\text{OFF} \\ 0, & \text{if } S_3\text{OFF and } S_6\text{ON} \end{cases}
\end{aligned} \tag{1}
$$

The expression for the output voltage space vector is given by (2)

$$v_i = \frac{2}{3}(v_{aN} + \alpha v_{bN} + \alpha^2 v_{cN}) \tag{2}$$

where $\alpha = e^{j(2\pi/3)}$ and $v_{aN} = S_a \cdot V_{in}$, $v_{bN} = S_b \cdot V_{in}$, $v_{cN} = S_c \cdot V_{in}$, respectively.

Based on the switch configuration, the possible voltage vectors in the $\alpha\beta$ frame are shown in Table 1. Corresponding to Table 1, the possible voltage vectors are represented in the complex plane in Fig. 3.

2.2. Dynamic modelling of the load side LC filter

The circuit diagram of VSI is depicted in Fig. 2. The model of LC output filter is described in (3) as follows:

$$
\begin{cases}
L_f \frac{di_f}{dt} = v_i - v_o - R_f i_f \\
C_f \frac{dv_o}{dt} = i_f - i_o
\end{cases} \tag{3}
$$

where L_f, R_f and C_f are the filter inductance, parasitic resistances of the inductor and the filter capacitance, respectively. The i_f, i_o and v_o are the measured quantities that represent the current through L_f, load current, and output voltage across the C_f respectively. while the voltage vector v_i is analytically calculated. The system continuous-time state-space equation can be written as:

$$\frac{dx(t)}{dt} = \mathbf{A}x(t) + \mathbf{B}v_i(t) + \mathbf{D}i_o(t) \tag{4}$$

where,

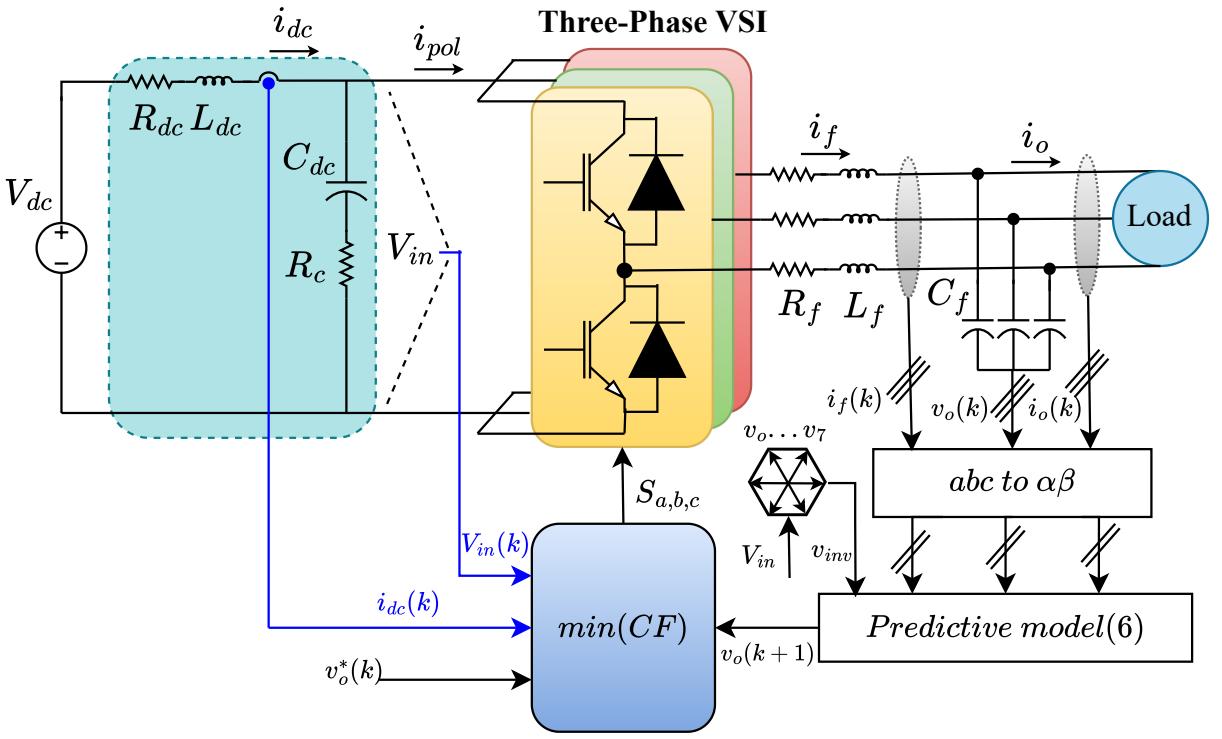


Fig. 4: FCS-MPC based stabilisation control scheme

$$\begin{cases} x = \begin{bmatrix} i_f \\ v_o \end{bmatrix}, \mathbf{A} = \begin{bmatrix} -\frac{R_f}{L_f} & -\frac{1}{L_f} \\ \frac{1}{C_f} & 0 \end{bmatrix} \\ \mathbf{B} = \begin{bmatrix} \frac{1}{L_f} \\ 0 \end{bmatrix}, \mathbf{D} = \begin{bmatrix} 0 \\ \frac{1}{C_f} \end{bmatrix} \end{cases} \tag{5}$$

The implementation of FCS-MPC relies on the discrete state-space model derived from the system dynamics. This discrete state-space model is obtained through the discretization of (4) using a sampling time T_s:

$$x(k+1) = \mathbf{A_d} x(k) + \mathbf{B_d} v_i(k) + \mathbf{D_d} i_o(k) \tag{6}$$

where,

$$\begin{cases} \mathbf{A_d} = e^{\mathbf{A}T_s} \\ \mathbf{B_d} = \int_0^{T_s} e^{\mathbf{A}\tau} \mathbf{B} d\tau \\ \mathbf{D_d} = \int_0^{T_s} e^{\mathbf{A}\tau} \mathbf{D} d\tau \end{cases} \tag{7}$$

To maintain precise tracking of the voltage reference, the cost function is formulated as follows [5]:

$$CF_{AC} = CF_1 + \zeta_{der} CF_2 + I_{lim} \tag{8}$$

$$\begin{cases} CF_1 = (v_o^*(k) - v_o(k+1))^2 \\ CF_2 = (\omega C_f v_{o\beta}^*(k) - i_{f\alpha}(k+1) + i_{o\alpha}(k+1))^2 \\ \quad +(\omega C_f v_{o\alpha}^*(k) + i_{f\beta}(k+1) - i_{o\beta}(k+1))^2 \\ I_{\text{lim}} = \begin{cases} \infty & , \text{ if } \left|i_{f,k+1}\right| > I_{\max} \\ 0 & , \text{ if } \left|i_{f,k+1}\right| \le I_{\max} \end{cases} \end{cases} \tag{9}$$

CF_1 is responsible for tracking the AC voltage. In [5], it was demonstrated that including CF_2 significantly improves steady-state performance. I_{lim} is implemented for limiting over current.

2.3. Dynamic modeling of the EMI filter

As illustrated in Fig. 2, the DC-side EMI LC filter is made up of an inductance L_{dc} in series with resistance R_{dc} and a capacitance C_{dc} with equivalent series resistance R_c. The dynamic equation for direct current link can be expressed as:

$$C_{dc}\frac{dV_{in}}{dt} = i_{dc} - i_{pol} \tag{10}$$

where i_{dc} is the measured value while i_{pol} can be expressed as

$$i_{pol} = S_a i_{fa} + S_b i_{fb} + S_c i_{fc} \tag{11}$$

S_a to S_c denote the switching signals for the respective phases. The predicted voltage $v_{in}(k+1)$ is calculated from (12) using the forward Euler approximation [5], expressed as follows:

$$V_{in}(k+1) = \frac{T_s}{C_{dc}}(i_{dc}(k) - i_{pol}(k)) + V_{in}(k) \tag{12}$$

2.4. Stabilization via FCS-MPC based cost function

To maintain precise tracking of the voltage reference while simultaneously stabilizing the DC-side voltage oscillations, the corresponding expression for the cost function is as follows [5]:

$$CF = CF_{AC} + \zeta_{dc} CF_{DC} \tag{13}$$

Where $CF_{DC} = (V_{in}^* - V_{in}(k+1))^2$ is employed for stabilizing DC-side voltage oscillations. The FCS-MPC implementation is shown in Fig. 4, with parameters given in Table 2. The corresponding simulation results are depicted in Fig. 5.

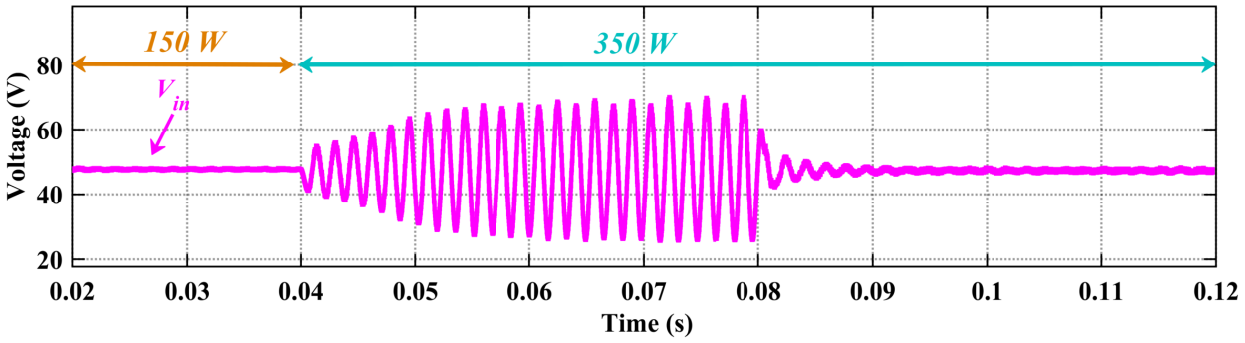
Table 2
Parameters for FCS-MPC simulation

T_s (μs)	V_{dc} (V)	v_o (V)	L_{dc} (μH)	C_{dc} (μF)	R_{dc} (Ω)	R_C (Ω)	Load (W)	L_f (mH)	C_f (μF)	R_f (Ω)	ζ_{der}, ζdc
15	48	19.6	300	220	0.1	0.05	150-350	1	25	0.1	0.5, 0.01

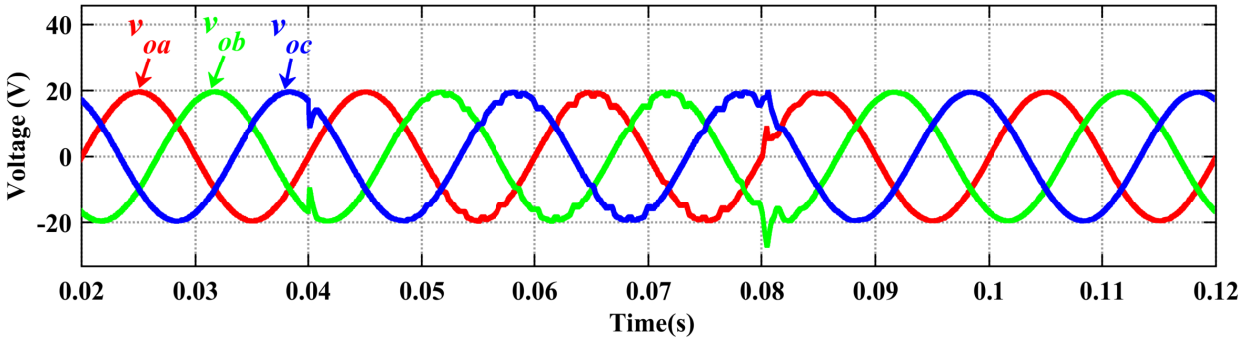
As presented in Fig. 5(a), when the power increases from 150 W to 350 W at $t = 0.04$ s, oscillations appear in V_{in} because only CF_{AC} of (8) was active. At $t = 0.08$ s, (13) is enabled, which damp the oscillations. The corresponding AC-side voltage and current are shown in Figs. 5(b) and 5(c).

3. Proposed ANN-Based Control Strategy

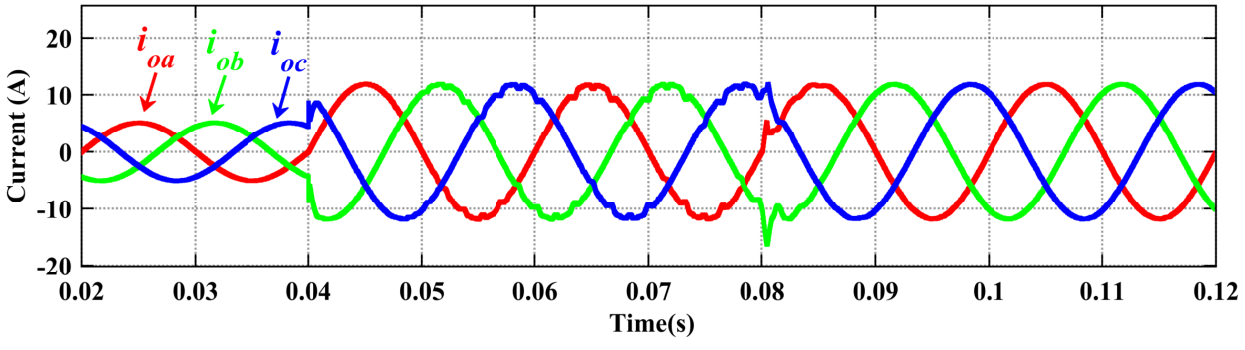
This section provides a concise introduction to the implementation of artificial neural networks (ANNs). The ANN implementation is discussed in four stages: 1) ANN architecture; 2) Data generation for ANN training; 3) ANN training; 4) ANN implementation. In modern power electronics systems, ANN plays a key role because of its better response due to the absence of dynamics. The following subsection discusses the ANN architecture adopted in this work.



(a)



(b)



(c)

Fig. 5: Simulation results with FCS-MPC: (a) DC voltage V_{in}, (b) AC output voltage v_{ox}, (c) Output current i_{ox}.

3.1. ANN Architecture

This work adopts a feed-forward ANN architecture. The output of each perceptron of every hidden layer in a feed-forward ANN can be mathematically modelled as a non-linear, cascaded, many-one function, comprising piecewise linear units as shown in (14).

$$\Psi^{i,k}(x^{i,k}) = f(b^{i,k} + \sum_{m \in \varepsilon_k} \omega_m^{i,k} x_m^{i,k}) \quad \forall i \in \varepsilon_L \tag{14}$$

Where, $\Psi^{i,k}$ depicts the output of the k^{th} perceptron of the i^{th} layer in a feed-forward neural network; $x^{i,k}$ depicts the set of inputs to the k^{th} perceptron of the i^{th} layer in the network; $f(\cdot)$ is the non-linear activation function; $b^{i,k}$ depicts the bias added at the k^{th} perceptron of the i^{th} layer; ε_k depicts the set of all the nodes of $(i-1)^{th}$ layer connecting

Table 3
Hyperparameters Used for Neural Network Training

Hyperparameter	Value	Description
Learning Rate	0.001	Controls how much the model is adjusted with respect to the loss gradient during training.
Batch Size	64	Number of training samples used in one forward/backward pass.
Number of Epochs	742	Total iterations over the entire training dataset.
Number of Layers	4	Total layers in the neural network, including input and output layers.
Layer Size	[10,10,10,7]	Number of neurons in each layer of the network.
Optimizer	Scaled Conjugate Gradient	Optimization algorithm used to update model weights.
Loss Function	Cross Entropy	Measures the performance of the classification model whose output is a probability value between 0 and 1.
Activation Function	ReLU	Introduces non-linearity in the model, allowing it to learn more complex patterns.
Weight Initialization	Xavier	Method for initializing the weights to improve convergence speed and performance.
Learning Rate Decay	0.9	Factor by which the learning rate is reduced over time.
Regularization	L2 (0.001)	Adds a penalty term to the loss function to reduce overfitting.

to the k^{th} perceptron of the i^{th} layer in the network; $\omega_m^{i,k}$ depicts the weight multiplied to the k^{th} perceptron of the i^{th} layer of the ANN; and ε_L depicts the set of hidden layers in the ANN.

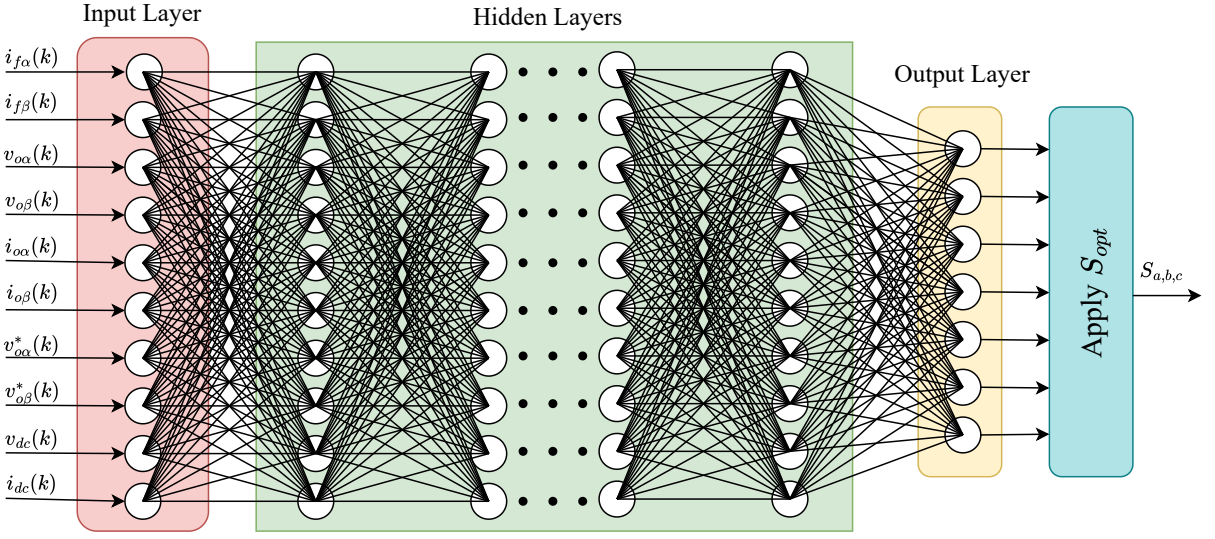


Fig. 6: Schematic representation of the ANN-based control strategy

Fig. 6 illustrates a shallow (single hidden layer) fully connected network with a multi-layer perceptron architecture, implemented in this work. The size of the hidden layer is optimised to give optimal results by the grid search algorithm. Table 3 presents the hyperparameters and activation functions used in the adopted ANN-based methodology.

3.2. Data Generation for ANN Training

Data from FCS-MPC-based simulations is collected for training the ANNs model. This section articulates the process for data generation. This data is used to train the adopted ANN architecture in accord with MPC implementation, to be discussed in Section 3.4. Also, the input and output data are selected as per the adopted ANN architecture as well. Here, $D_{in} = (i_{f\alpha}(k), i_{f\beta}(k), v_{o\alpha}(k), v_{o\beta}(k), i_{o\alpha}(k), i_{o\beta}(k), v^*_{o\alpha}(k), v_{dc}(k), i_{dc}(k))$ denotes the set of

inputs to the ANN; $\mathcal{D}_{targ} = (\upsilon_1, ..., \upsilon_7)$ denotes the selected targets for the ANN training (see Table 1). Aforementioned inputs and targets are obtained by running the simulations in iterative manner with different parameter scenarios. In this regard, Table 4 presents the chronological variations in the system parameters, considered in this work. To significantly eliminate the possibility of outliers during the real-time implementation of the proposed method, in Table 4, thirty-seven different data generation scenarios are considers. Subsequently, as part of data generation, the dc-side EMI filter parameters are varied along with ac-side LC filter parameter variation. Moreover, to test the robustness of the proposed system, in implementation, both input voltage (V_{dc}) and output side reference voltage (υ_o) variations are considered along with the load variations.

3.3. ANN Training

The aforementioned generated data is used to train the ANN. The aim of ANN training is to reduce the root mean square loss ($\mathcal{L}_{RMS}$) between the output of ANN and the generated, labelled target data, shown in (15). where j denotes the output layer of the ANN; $|\cdot|$ denotes the cardinality of the set; $\hat{\psi}^{j,k}$ denotes the target value of output at the k^{th} perceptron of the output layer of the ANN. To effectively minimise the loss, the generated dataset is divided into three sub-datasets, namely, training data, validation data, and test data. Here, the training data is used to train the ANN; the validation data is used to validate the performance of the ANN after training; and the test dataset is used to evaluate the performance. The training data is further divided into randomly selected batches of equal size. Doing so reduces the chances of the occurrence of distribution shifts in supervised learning. During the training, the weights and the biases get updated through back-propagation for the average loss over each batch of training data. The epoch is defined as when a whole data set is iterated over once with the help of several batches. The mathematical expressions to update the weights and biases for the k^{th} perceptron of i^{th} layer through back propagation are shown in (16) and (17), respectively. Here, $\mathfrak{P}$ depicts the learning rate.

$$\mathcal{L}_{RMS} = \sqrt{\frac{1}{\left|\varepsilon_j\right|} \sum_{k \in \varepsilon_j} \left(\psi^{j,k} - \hat{\psi}^{j,k}\right)^2} \tag{15}$$

$$\omega_{t+1}^{i,k} = \omega_t^{i,k} - \mathfrak{P} \frac{\partial \mathcal{L}_{RMS}}{\partial \omega^{i,k}} \tag{16}$$

$$b_{t+1}^{i,k} = b_t^{i,k} - \mathfrak{P} \frac{\partial \mathcal{L}_{RMS}}{\partial b^{i,k}} \tag{17}$$

$$transig(z^i) = \frac{2}{1 + e^{-2(z^i)}} - 1 \tag{18}$$

$$ReLU(z^i) = \max(0, z^i) \tag{19}$$

Algorithm 1 gives the chronological details for the ANN training procedure. Step-**1** accounts for importing the training subset of input data and the target dataset for the ANN training. Step-**2** denotes the random initialization of the weights and biases for the adopted ANN architecture. Step-**3** encompasses the iterative process for the defined limit on the number of episodes. For each iteration in Step-**3**, another iterative loop, shown in Step-**4**, is defined that takes random samples of fixed batch size from the training dataset to update the ANN. Step-**5** calculates the ANN output $\left(\psi_{t,\zeta}^{j,k}\right)$ for the given input sampled batch. Note that the size of the obtained output would be the same as the batch size as well. Step-**6** obtains a scalar, scaled RMS loss between $\left(\psi_{t,\zeta}^{j,k}\right)$ and the target output $\left(\hat{\psi}_{t,\zeta}^{j,k}\right)$ for the sampled input. If the obtained loss is within the tolerance limit (ϵ), the Steps-**7,8,10** lead to the end of the training. Otherwise, the gradients are computed and back-propagated through gradient descent to update the weights and biases of the network, Step-**9**. Finally, in Step-**11**, the trained ANN network with the desired error tolerance is obtained at the output.

The proposed method utilises a feed-forward shallow-type neural network with ten input features and seven output features. The generated data is fed to the network with randomly initialised weights (ω) to ensure the global convergence

of the training. The ANN training uses the "$transig$," or hyperbolic tangent activation function (18), to learn and imitate the non-linearity present in the FSC MPC-based controller while avoiding the vanishing gradient issue with standard non-linear activation functions, viz. "$ReLU$," or rectified linear unit (19). Here, z^i represents the vector containing the inputs to the activation function at i^{th}.

Table 4
Description of the dataset for training

Sample No.	T_s (μs)	V_{dc} (V)	v_o (V)	L_{dc} (μH)	C_{dc} (μF)	R_{dc} (Ω)	R_C (Ω)	Load (W)	L_f (mH)	C_f (μF)	R_f (Ω)	Cycle/Samples
1	10	48	19.6	300	220	0.1	0.05	**50**	1	25	0.1	3/6001
2	10	48	19.6	300	220	0.1	0.05	**100**	1	25	0.1	3/6001
3	10	48	19.6	300	220	0.1	0.05	**150**	1	25	0.1	3/6001
4	10	48	19.6	300	220	0.1	0.05	**200**	1	25	0.1	3/6001
5	10	48	19.6	300	220	0.1	0.05	**250**	1	25	0.1	3/6001
6	10	48	19.6	**250**	**180**	0.1	0.05	**300**	1	25	0.1	3/6001
7	10	48	19.6	300	220	0.1	0.05	350	1	25	0.1	3/4001
8	15	48	19.6	300	220	0.1	0.05	**50**	1	25	0.1	3/4001
9	15	48	19.6	300	220	0.1	0.05	**100**	1	25	0.1	3/4001
10	15	48	19.6	300	220	0.1	0.05	**150**	1	25	0.1	3/4001
11	15	48	19.6	300	220	0.1	0.05	**200**	1	25	0.1	3/4001
12	15	48	19.6	300	**200**	0.1	0.05	**250**	1	25	0.1	3/4001
13	15	48	19.6	**330**	**200**	0.1	0.05	**300**	1	25	0.1	3/4001
14	15	48	19.6	300	**200**	0.1	0.05	350	1	25	0.1	3/4001
15	20	48	19.6	300	**200**	0.1	0.05	**50**	1	25	0.1	4/4001
16	20	48	19.6	**330**	220	0.1	0.05	**100**	1	25	0.1	4/4001
17	20	48	19.6	**330**	220	0.1	0.05	**150**	1	25	0.1	4/4001
18	20	48	19.6	**330**	220	0.1	0.05	**200**	1	25	0.1	4/4001
19	20	48	19.6	**360**	220	0.1	0.05	**250**	1	25	0.1	4/4001
20	20	48	19.6	**360**	**200**	0.1	0.05	**300**	1	25	0.1	4/4001
21	25	48	19.6	**560**	**180**	0.1	0.05	350	1	**20**	0.1	4/3201
22	25	48	19.6	**560**	**180**	0.1	0.05	**100**	1	**20**	0.1	4/3201
23	25	48	19.6	**560**	**180**	0.1	0.05	**150**	1	**20**	0.1	4/3201
24	25	48	19.6	**560**	**180**	0.1	0.05	**200**	1	**20**	0.1	4/3201
25	25	48	19.6	**560**	**180**	0.1	0.05	**250**	1	**20**	0.1	4/3201
26	25	48	19.6	**560**	**180**	0.1	0.05	**300**	1	**20**	0.1	4/3201
27	25	48	19.6	**560**	**180**	0.1	0.05	350	1	**20**	0.1	4/3201
28	10	48	19.6	300	220	0.1	0.05	**100**	**2**	25	0.1	3/6001
29	10	48	19.6	300	220	0.1	0.05	**150**	**2**	25	0.1	3/6001
30	10	48	19.6	300	220	0.1	0.05	**200**	**2**	**40**	0.1	3/6001
31	10	48	19.6	300	220	0.1	0.05	**250**	**2**	**40**	0.1	3/6001
32	10	48	19.6	300	200	0.1	**0.06**	350	1	25	0.1	3/6001
33	10	48	19.6	300	220	**0.11**	0.05	350	1	25	0.1	3/6001
34	10	48	**17**	300	220	0.1	0.05	350	1	25	0.1	3/6001
35	10	48	**21**	300	220	0.1	0.05	350	1	25	0.1	3/6001
36	10	**51**	19.6	300	220	0.1	0.05	350	1	25	0.1	3/6001
37	10	**45**	19.6	300	220	0.1	0.05	350	1	25	0.1	3/6001

The evaluation of the training performance is illustrated in Fig. 7 through a confusion matrix. The model achieved its best validation error of 0.10523 at epoch 742. Overall, the system demonstrated an accuracy of 72.60% when tested on a dataset comprising 176437 instances.

3.4. ANN Implementation

This subsection discusses the online implementation of offline training based ANN. Fig. 8 shows the implemented architecture for the proposed ANN-based method. In Fig. 8, the trained ANN is used as the controller, in place of MPC, when compared to Fig. 4. Being a data-driven control strategy, this method reduces the online computation

Algorithm 1: Offline Training

Input: Training and Target ($\hat{\psi}^{j,k}\ \forall i \in \varepsilon_L, k \in \varepsilon_i$) data;
Initialize: $\omega_0^{i,k}, b_0^{i,k} \quad \forall i \in \varepsilon_L, k \in \varepsilon_i$;
for *t = 1, 2, 3...Max number of epochs* **do**
  for ζ *= 1, 2, 3...Number of batches* **do**
    Compute: $\psi_{t,\zeta}^{j,k}$ through feed-forwarding the input sample;
    Obtain:

$$\mathcal{L}_{RMS} = \frac{\sum_{B} \sqrt{\frac{1}{|\varepsilon_j|} \sum_{k \in \varepsilon_j} \left(\psi^{j,k} - \hat{\psi}^{j,k}\right)^2}}{|B|}$$

    if $\mathcal{L}_{RMS} \leq \epsilon$ **then**
      Set: $FLAG = 1$;
      Break the loop;
    Update: The weights and biases through back propagation using gradient descent

$$\omega_{t,\zeta}^{i,k} = \omega_{t,\zeta-1}^{i,k} - \mathfrak{P}\frac{\partial \mathcal{L}_{RMS}}{\partial \omega^{i,k}}$$

$$b_{t,\zeta}^{i,k} = b_{t,\zeta-1}^{i,k} - \mathfrak{P}\frac{\partial \mathcal{L}_{RMS}}{\partial b^{i,k}} \quad \forall i \in \varepsilon_L, k \in \varepsilon_i;$$

  if *FLAG = 1* **then**
    Break the loop;
Output: Trained ANN for implementation;

burden and circumvents the lags introduced by the MPC based controllers. Furthermore, this approach makes the stabilization control as a parameter free stabilization control strategy. The switching signals are the outputs of the proposed controller. Hence, proposed approach does not need any modulator either. To verify the aforementioned claims, points claimed in Section 1.2, the following section rigorously assesses the system performance with state-of-the-art conventional strategies.

4. Simulation Results and discussion

In this section, the simulation results of both the FCS-MPC and ANN-based controllers are presented. The parameters used for the simulation system are comprehensively detailed in Table 2. To ensure accurate comparisons, the ANN-based control strategy is implemented using MATLAB/Simulink as illustrated in Fig. 8. This allowed us to directly compare its performance against the established FCS-MPC method under identical conditions.

The comparative simulation results from both methodologies are visually represented in Fig. 9. Upon careful examination of these results, it becomes evident that the proposed ANN-based method demonstrates superior performance in two critical aspects: settling time and THD. It exhibits a faster settling time, reaching steady-state operation more quickly than its FCS-MPC counterpart which is 7 ms as shown in Fig. 9(b). Additionally, the ANN approach consistently achieves lower THD values which is 0.81% as shown in Fig. 9(d). These improvements in THD and settling time highlight the potential of ANN-based controllers offering a promising alternative to FCS-MPC control methods.

4.1. Robustness performance analysis Under Parameter Variations

To evaluate the robustness of the proposed control scheme against parametric variations, a $\pm$50% variation in both L_f and C_f of the filter has been considered. As shown in Table 5, these variations impact the THD of the load voltage v_{ox}, as well as the settling time V_{in}. In both scenarios, the ANN-based method demonstrates superior performance compared to other approaches.

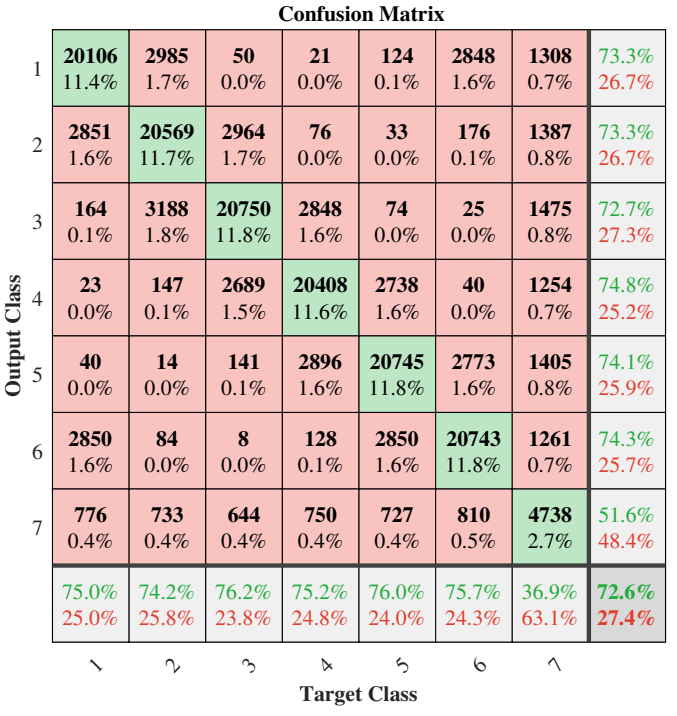


Fig. 7: Training confusion Matrix

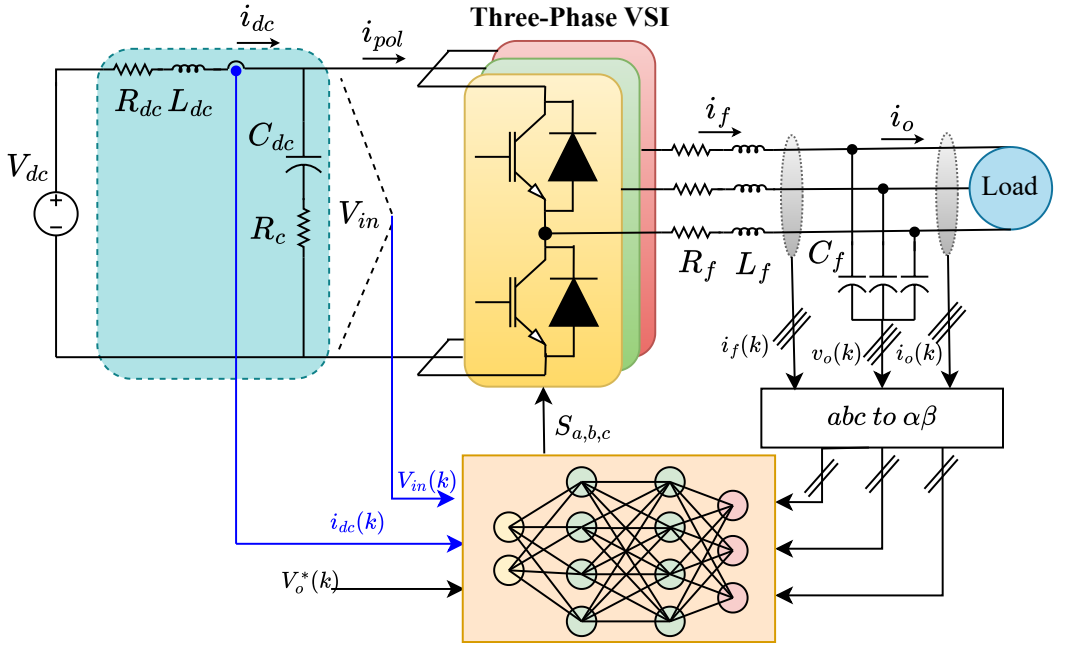


Fig. 8: Schematic representation of the ANN-based control strategy

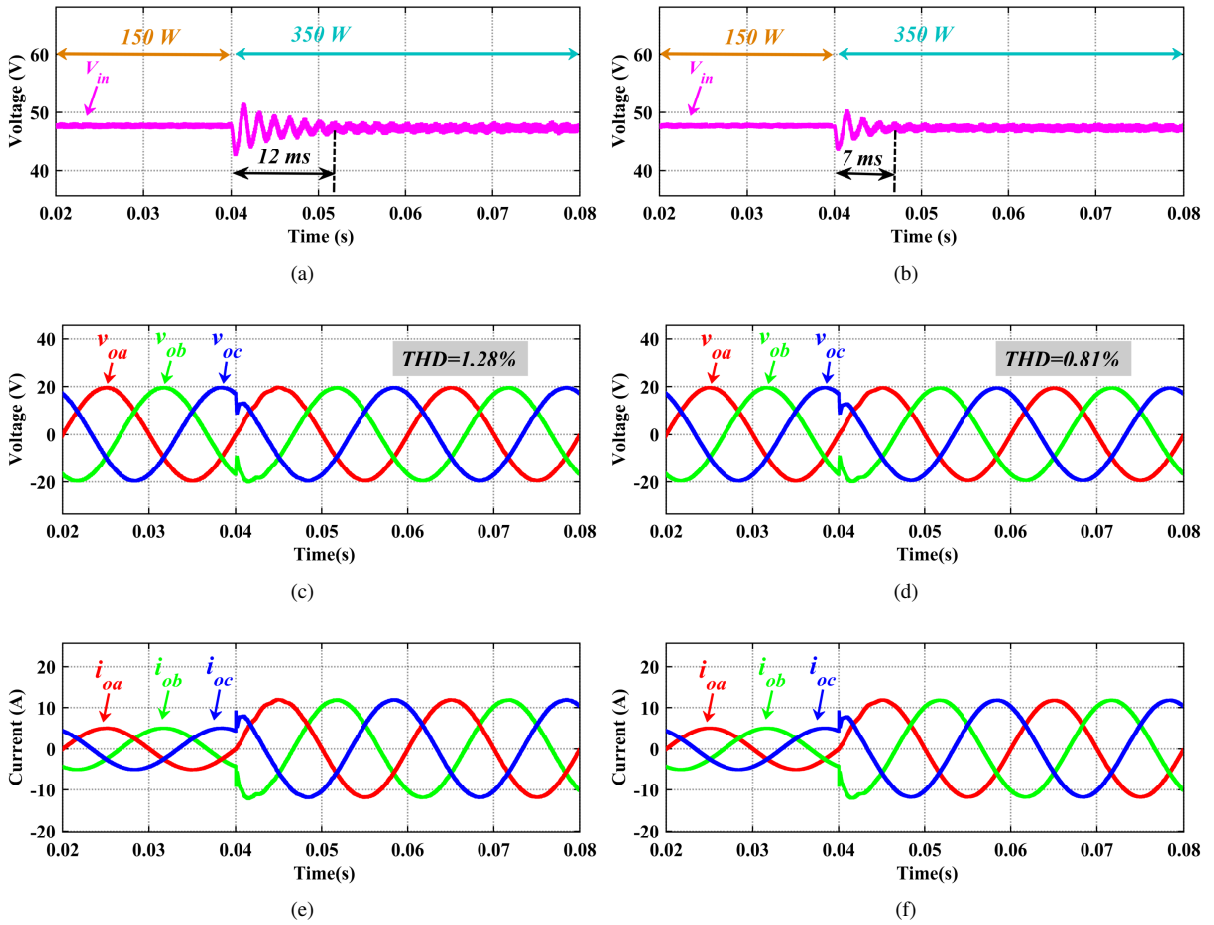


Fig. 9: Simulation results with FCS-MPC (a) DC voltage V_{in}, (c) AC output voltage v_{ox}, (e) Output current i_{ox}, and with ANN(b) DC voltage V_{in}, (d) AC output voltage v_{ox}, (f) Output current i_{ox} respectively.

Table 5
Robustness Performance of Proposed Control Strategy Under Parametric Variations

Parameter Variation	**(FCS-MPC)** [5]		**Proposed ANN Based control**	
	Settling time (ms)	**THD of ac voltage (%)**	**Settling time (ms)**	**THD of ac voltage (%)**
-50% L_f, -50% C_f	17.0	1.91	11.0	1.22
+50% L_f, -50% C_f	12.2	1.43	7.5	0.93
+50% L_f, +50% C_f	10.0	0.97	6.0	0.73
-50% L_f, +50% C_f	14.0	1.56	9.0	1.03

5. Conclusion

This study demonstrates the advantages of the proposed ANN-based control method over FCS-MPC for voltage oscillations stabilization in DC microgrids. The ANN approach achieves superior performance with a faster settling time of 7 ms and lower THD of 0.81%. It effectively addresses parameter mismatch issues through a parameter-free strategy, improves AC-side voltage quality, and swiftly suppresses voltage oscillations. The offline training of the ANN significantly reduces online computational burden, making it more efficient for real-time applications. These

improvements in performance, power quality, and computational efficiency highlight the potential of ANN-based controllers as a promising alternative to FCS-MPC, paving the way for more robust and adaptable systems across various applications.

References

[1] H. Liao, X. Zhang, H. Ma, Impedance-shaping-based stability control of point-of-load converter integrated with emi filter in dc microgrids, IEEE Access 10 (2021) 25034–25043.

[2] Z. Ma, X. Zhang, J. Huang, B. Zhao, Stability-constraining-dichotomy-solution-based model predictive control to improve the stability of power conversion system in the mea, IEEE Transactions on Industrial Electronics 66 (2018) 5696–5706.

[3] R. Kumar, C. Bhende, A virtual adaptive rc damper control method to suppress voltage oscillation in dc microgrid, International Journal of Electrical Power & Energy Systems 146 (2023) 108795.

[4] R. Kumar, C. N. Bhende, Active damping stabilization techniques for cascaded systems in dc microgrids: A comprehensive review, Energies 16 (2023) 1339.

[5] T. Dragičević, Dynamic stabilization of dc microgrids with predictive control of point-of-load converters, IEEE Transactions on Power Electronics 33 (2018) 10872–10884.

[6] M. A. Hassan, C.-L. Su, J. Pou, G. Sulligoi, D. Almakhles, D. Bosich, J. M. Guerrero, Dc shipboard microgrids with constant power loads: A review of advanced nonlinear control strategies and stabilization techniques, IEEE Transactions on Smart Grid 13 (2022) 3422–3438.

[7] M. N. Hussain, G. Melath, V. Agarwal, An active damping technique for pi and predictive controllers of an interlinking converter in an islanded hybrid microgrid, IEEE Transactions on Power Electronics 36 (2020) 5521–5529.

[8] X. Liu, L. Qiu, Y. Fang, J. Rodríguez, Predictor-based data-driven model-free adaptive predictive control of power converters using machine learning, IEEE Transactions on Industrial Electronics 70 (2022) 7591–7603.

[9] I. S. Mohamed, S. Rovetta, T. D. Do, T. Dragicević, A. A. Z. Diab, A neural-network-based model predictive control of three-phase inverter with an output *lc* filter, IEEE Access 7 (2019) 124737–124749.

[10] D. Wang, Z. J. Shen, X. Yin, S. Tang, X. Liu, C. Zhang, J. Wang, J. Rodriguez, M. Norambuena, Model predictive control using artificial neural network for power converters, IEEE Transactions on Industrial Electronics 69 (2021) 3689–3699.

[11] T. Dragičević, M. Novak, Weighting factor design in model predictive control of power electronic converters: An artificial neural network approach, IEEE Transactions on Industrial Electronics 66 (2018) 8870–8880.